



Fachhochschul-Bachelorstudiengang
SOFTWARE ENGINEERING
A-4232 Hagenberg, Austria

Creating a Visual Studio Extension to Support the Development of Discord Bots with Discord.Net

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Stefan Christian Merwa

(Version 2)

Begutachtet von FH-Prof. DI Johann Heinzelreiter

Hagenberg, Februar 2024

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere.

This printed thesis is identical with the electronic version submitted.

Date

Signature

1 Abstract

Verwenden Sie die „documentclass“ „article“? Ich denke „report“ ist für eine BA besser geeignet.

This bachelor's thesis introduces a Visual Studio Extension that is about making the creation of Discord bots a lot more comfortable with Discord.NET. The compelling part *... is how it uses/integrates ...* is it uses Microsoft Roslyn and the powerful tools within Visual Studio to give developers support with intelligent code completion through precise code analysis, and a direct line to Discord's API. This project fills a real need for better tools in bot making, aiming to make the whole process more efficient and user-friendly in Visual Studio.

2 Kurzfassung

In dieser Bachelorarbeit wird eine Visual-Studio-Extension vorgestellt, die die Erstellung von Discord-Bots mit Discord.NET vereinfachen soll. Das Besondere daran ist die Verwendung ^{von} Microsoft Roslyn und den effektiven Tools ^{von} Visual Studio, um Entwickler mit intelligenter Code-Vervollständigung durch präzise Code-Analyse und einer direkten Verbindung zur API von Discord zu unterstützen. Dieses Projekt ^{deckt} erfüllt einen echten Bedarf an besseren Tools für die Bot-Erstellung und zielt darauf ab, den gesamten Prozess in Visual Studio effizienter und benutzerfreundlicher zu gestalten. } sehr komplizierter Satz

Contents

1	Abstract	3
2	Kurzfassung	4
3	Introduction	8
4	Discord	9
4.1	Discord Bots	9
4.2	Discord.Net	10
5	Foundation	11
5.1	Visual Studio Extension Development	12
5.1.1	Anatomy	13
5.1.2	Commands	15
5.1.3	Displaying Custom Windows	20
5.1.4	Logging	21
5.2	MS Roslyn	22
5.2.1	Compiler phases	22
5.2.2	Code Analysis	23
5.2.3	Syntax Visualizer in Visual Studio	26
5.3	Editor API	27
5.3.1	Async Completion API	27
5.4	Managed Extensibility Framework	28
6	Building the Visual Studio Extension	30
6.1	VS Commands	32
6.2	MEF Components Project	37
6.3	Code Analyser	40
6.4	Potential result	44
7	Closing	47

List of Figures

1	Workflow overview to acquire the wanted solution	11
2	Visual Studio Installer	12
3	File structure of a VSIX Project	13
4	Design view of the .vsixmanifest file	13
5	.vsixmanifest file assets	14
6	Properties of a VSIX Command Table file	14
7	Result of the command definition	17
8	Custom output window pane	21
9	Compiler pipeline functional areas	22
10	Syntax Visualizer	26
11	References of the newly created Visual Studio Project	30
12	References of the newly created Visual Studio Project	31
13	Workflow for commands	32
14	Commands in project context	32
15	Command that configures how the data will be generated	33
16	Workflow for the MEF Project	37
17	Workflow for the implemented code analyser using Roslyn	41
18	Shows analysis workflow with the IDiscordCompletionEngine as start point	42
19	Completion items for getting a server	44
20	Committed server completion item	44
21	Completion items for getting a user that is inside the given server	45
22	Committed user completion item	46
23	Role suggestions in Linq method	46

Listings

1	Custom command group definition	16
2	Custom command group definition	16
3	Custom command button definition	16
4	Command placement	17
5	Possible command base	18
6	Base class for async command handling	18
7	Custom command	19
8	Calling command's InitializeAsync in package initialization	20
9	Displaying a custom window	20
10	Displaying a custom window	21
11	Roslyn Workspace	23
12	Syntax Example	24
13	Syntax Example Output	24
14	Semantics Example	25
15	Semantics Example Output	26
16	ICustomHandler for MEF example	28
17	CustomHandler Export for MEF example	28
18	ICustomHandler Import for MEF example	29

LISTINGS

19	DiscordServerImageConfig.json	33
20	Generating the data loaded from Discord's API	34
21	Implementation of the DiscordRestEntityService	35
22	Generated data	36
23	Discord Completion Source Provider	37
24	Discord Completion Source	38
25	Discord Completion Commit Manager Provider	39
26	Discord Completion Commit Manager	39
27	Getting the semantic model and syntax token	41
28	Code snippets from the context analyser	42

3 Introduction

In recent years, Discord has emerged as a popular platform not just for communication but for community engagement through custom bots. This thesis explores the simplification of the bot creation process using a Visual Studio Extension and Discord.NET. Discord.NET serves as a .NET library facilitating bot development, while Visual Studio is chosen for its comprehensive support and robust features.

The challenge of developing Discord bots lies in the complexity of Discord's API and the .NET framework, making the process daunting. This extension aims to bridge this gap with an intuitive interface and integrated development tools that streamline bot development. Features such as intelligent code suggestions and automatic error detection enhance productivity and reduce the development timeline.

Furthermore, this thesis utilizes Microsoft Roslyn, a compiler platform for real-time code analysis, to provide feedback on code quality and automate routine coding tasks. This introduction sets the stage for a detailed examination of the extension's development, including its architectural design, functionality, and the integration of underlying technologies. ✓

4 Discord

Discord, a by now popular voice, video, and text chat app, has significantly influenced the way people around the world communicate, especially within various communities and friend groups. Originally developed as a tool for gamers, it has moved on from its initial purpose to more a versatile platform for different conversations, ranging from casual chats to more structured discussions (Discord, 2022).

Origin

The application was founded by Jason Citron and Stanislav Vishnevskiy, who shared a passion for gaming and a vision for improved online communication. Citron, an American entrepreneur, founded OpenFeint, an early mobile gaming platform and Vishnevskiy, a Russian software engineer, had a background in projects like Google Hangouts and Yahoo! Messenger. Their combined expertise in software engineering and game design facilitated the creation of Discord, launched in May 2015 (TheEnlightenedMindset, 2023).

Architecture and Features

Discord is well-known for its server-based structure, which is the core of its functionality. These servers, customizable as public or private consist of voice and text channels. This structure enables many different communities, from study groups and book clubs to remote workers, to establish their purpose within the platform. Another significant aspect of Discord is its comprehensive moderation capabilities. Server administrators have the authority to manage user permissions, establish automated moderation protocols, and utilize features like word filtering (whitelist or blacklist) to maintain a safe and respectful environment for community interactions. Furthermore, Discord's ability to integrate smoothly with other well known platforms contributes to its growing popularity and effectiveness. This integration capability allows users to perform a variety of actions, such as streaming live gaming sessions on Twitch, sharing YouTube videos directly in chat channels, or collaborating on documents via Google Docs. These integrations play an important, if not the most important role in enhancing the overall utility and user experience of Discord (GameInfoHub, 2023).

4.1 Discord Bots

Bots are a key feature of Discord and a type of software that automates tasks and interacts with users in Discord. They are integrated into servers (4) and perform a variety of functions based on the commands and services like moderation, games, music, internet searches, and payment processing based on the programming (Agnihotri et al., 2022). Bots use Discord's Application Programming Interface to interact with the server

and its users. This API allows bots to send and receive messages, manage user roles, and more. It is important to note, that one bot can be used for as many servers as required.

Preparation

Discord bot development is a multifaceted process involving understanding the Discord platform, using its APIs, managing bot accounts, and writing and troubleshooting code. Developers must consider aspects like bot complexity, security, permissions, and community engagement. Utilizing available resources, best practices, and community support is key to creating successful and effective Discord bots (ToptalDevelopers, 2022). Before developing the bot, it first needs to be created through the Discord Developer Portal. There is an option to create a new application, which then will contain the bot user that can be configured with various settings like permissions and also provides a token for establishing a connection to Discord's API.

Development

Developing a Discord bot involves choosing a programming language and corresponding library that interfaces with the Discord API. Each language offers unique libraries or frameworks, designed to simplify the bot development process by abstracting the Discord API's complexities. To name a few, amongst the most popular ones are *Discord.js* for Javascript, *discord.py* for Python, *JDA* for Java, *DiscordGo* for Go and *Discord.Net* for C# . The choice of language and library depends on the developer's familiarity with the language, the specific requirements of the bot, and the desired level of abstraction from the Discord API. Each language and library has its advantages and is suited to different types of projects and developer preferences.

4.2 Discord.Net

Discord.Net is an asynchronous, multi-platform .NET Library designed to interface with the Discord API. It is an unofficial .NET API wrapper for the Discord client and also very feature rich by supporting slash commands, interaction frameworks and message components. To get started with Discord.Net, a good understanding of advanced .NET programming concepts like the Task-based Asynchronous Pattern is required, since the library extensively uses them. There are a lot of tutorials and also a comprehensive guide that provides detailed instructions and examples to help developers (Discord.Net, 2015). There exists a public Discord server for Discord.Net with more than 3000 current members, some of whom are willing to offer support and answer questions about the library. For common C# questions there also exists a Discord server with almost 50.000 members, where there is a channel dedicated to Discord development.

5 Foundation

To achieve the wanted capabilities for the Visual Studio Extension, multiple technologies are required. First a core understanding of how Visual Studio Extensions are developed and what features and possibilities they bring to the table is necessary. To effectively implement this process, it's essential to first set up a way to retrieve data from the Discord API. Since it's common to develop just one bot per C# project, it makes sense to trigger the process for each individual project. The way of implementation will be explained in more detail later. This data is then used for code completion utilizing the Editor API. Additionally, using the said API requires working with the Managed Extensibility Framework (MEF), since the Editor API depends on it. An important step in this process is figuring out exactly when and what type of data should be made available for code completion, and this is where MS Roslyn comes in, offering precise code analysis to make these determinations.

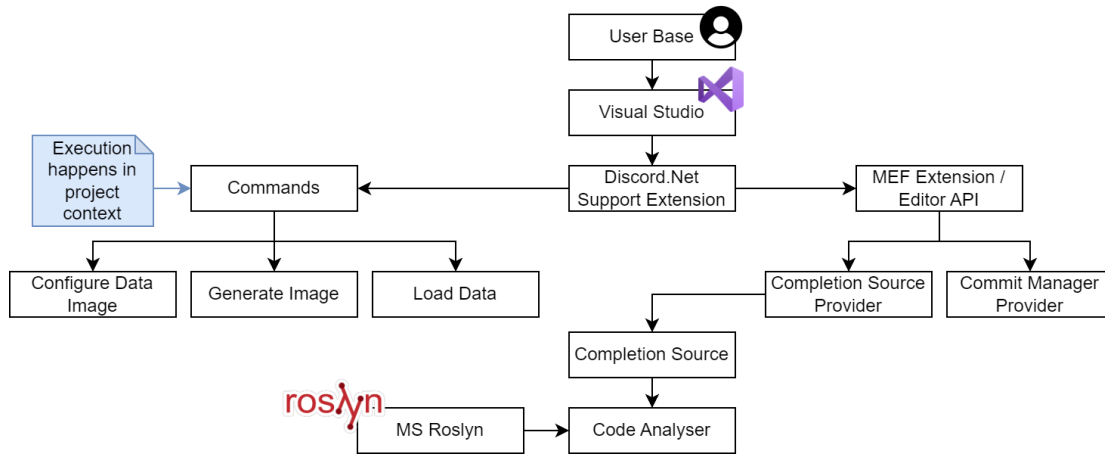


Figure 1: Workflow overview to acquire the wanted solution

To clarify Figure 1 in a more detailed manner, imagine a user who is coding in Visual Studio and has installed an extension that provides the necessary features mentioned in the Introduction (Section 3). This extension includes various Commands that can be accessed by right clicking a project. For interacting with the Editor API, a separate project is created and then added to the main project as a MEF component. For reference, it would also be possible to create the MEF components in the main project, but in terms of readability and maintainability it makes sense to put them in a separate project. And that is where the user's code is analyzed using MS Roslyn. The details of these steps are thoroughly explained in the following sections.

5.1 Visual Studio Extension Development

To participate in Visual Studio Extensibility (VSIX), the workload needs to be installed.

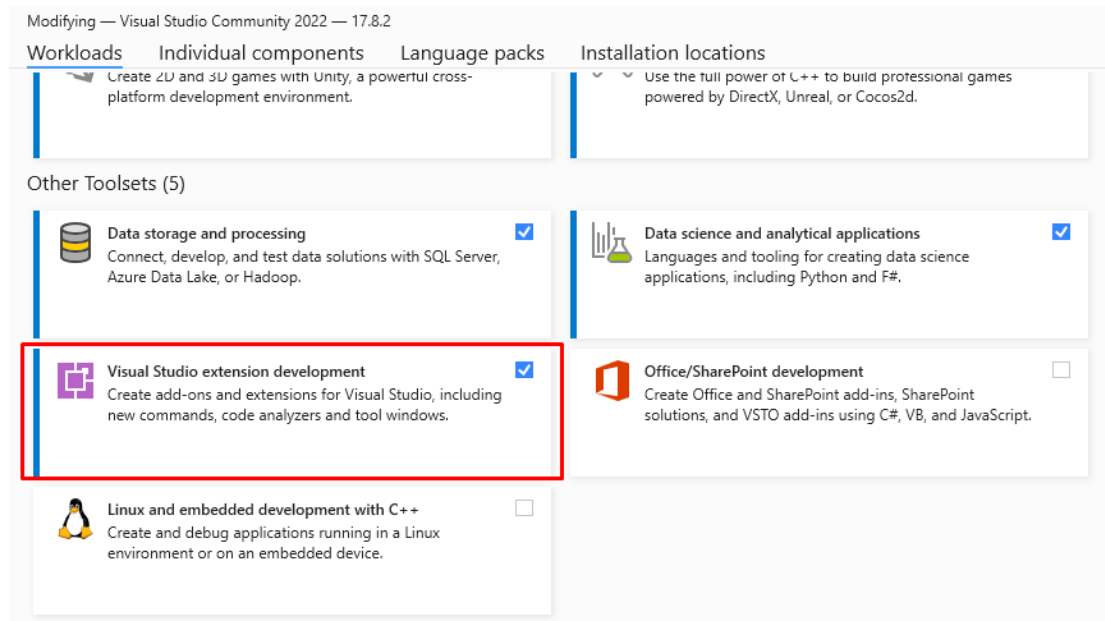


Figure 2: Visual Studio Installer

The types of extensions are quite varied. They can range from supporting new languages not included in Visual Studio, to productivity tools that enhance the core IDE experience, to domain-specific designers for specific scenarios like data design or cloud support. In Visual Studio, many different parts can be extended with the most common features to be commands, menus, toolbars, windows, IntelliSense, and projects (Microsoft, 2023). For example, there exists the Visual Studio Marketplace, where many Extensions are available for free or are even open sourced.

immer

5.1.1 Anatomy

File Structure

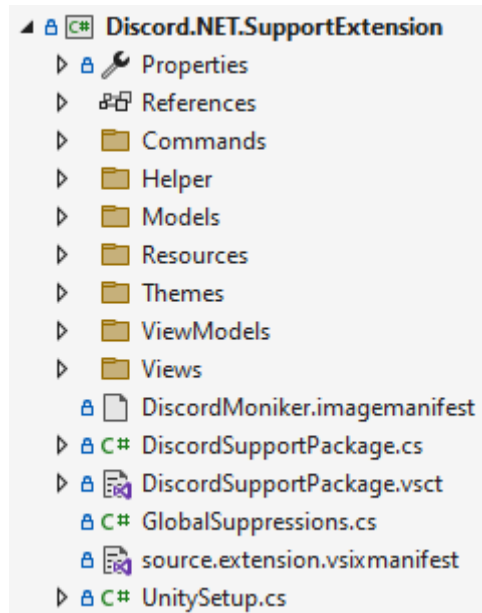


Figure 3: File structure of a VSIX Project

← kann man das tatsächlich der Abb. entnehmen

Figure 3 shows that the primary file in a Visual Studio extension project is the `.vsixmanifest` file (`source.extension.vsixmanifest`), an XML file that provides essential information about the extension to Visual Studio. It registers all the components of the extension and is the only required file in a VSIX project.

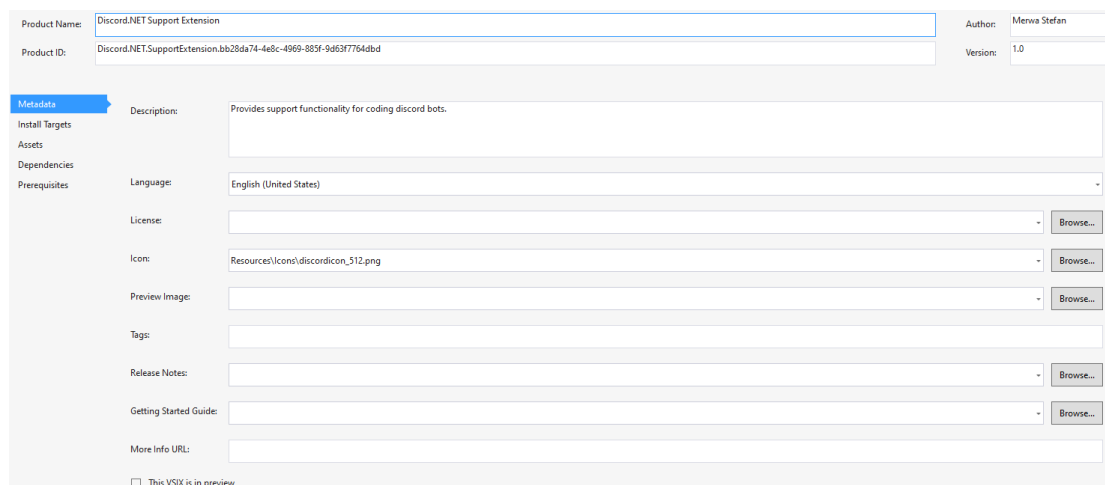


Figure 4: Design view of the `.vsixmanifest` file

Figure 4 shows the contents of the Manifest for a Visual Studio Extension. This

context menu pops up after double clicking on the file, so editing is made easier.

Metadata	Source	Type	Path	Target Version
Install Targets	Project	Microsoft.VisualStudio.VsPackage	Discord.NET.SupportExtension;PkgdefProjectOutputGroup	
Assets	Project	Microsoft.VisualStudio.MefComponent	Discord.NET.SupportExtension.Mef	
Dependencies	Project	Microsoft.VisualStudio.Assembly	Discord.NET.SupportExtension.Core	
Prerequisites				

Figure 5: .vsixmanifest file assets

As mentioned in Section 5 the interaction with the Editor API requires the usage of the MEF. That leads to the creation of a second project that will be imported to the main project as a MEF component. So a new asset is required, that goes with the source, type and path, as in this case the source is a *project*, the type is a *MefComponent* and the Path refers to the second project *Discord.NET.SupportExtension.Mef* which is illustrated in Figure 5. (Microsoft, 2021c)

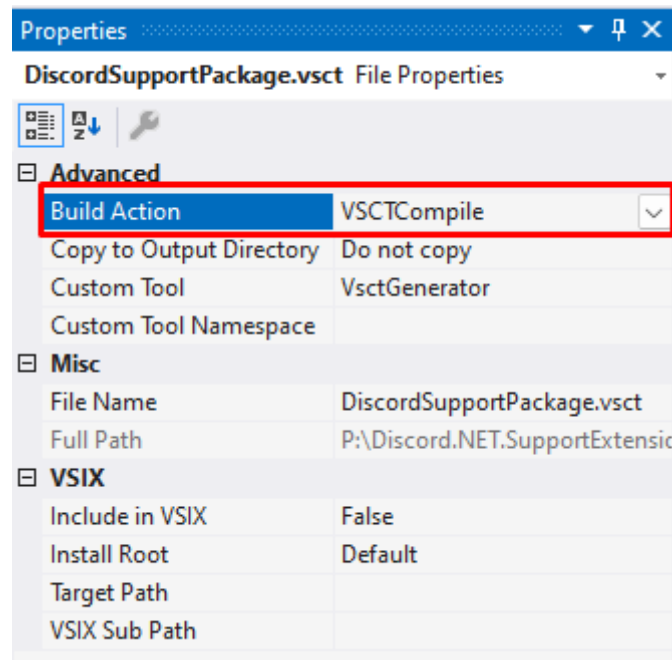


Figure 6: Properties of a VSIX Command Table file

Commands within the extension are defined in the command table (*DiscordSupportPackage.vsct* in Figure 3), another XML file. This file includes definitions for button commands, menus, and keyboard shortcuts. Figure 6 shows that its contents are compiled as an Assembly Resource, which Visual Studio utilizes to build its command table menu structure. However, this file only declares components and does not manage command executions. The Package.cs file (*DiscordSupportPackage.cs* in Figure 3), serves as

the primary entry point for most extensions. This file typically contains command handlers, tool windows, options pages, services, and other components. (Microsoft, 2021c)

Compilation

The project is compiled into a *.vsix* file, which can be found in either the */bin/debug* or */bin/release* folder, depending on the build configuration set for the current solution. The workload for Visual Studio extension development includes specialized MSBuild targets and tasks specifically designed for managing VSIX projects. During the build process of the VSIX project, it is automatically deployed to the Experimental Instance of Visual Studio. (Microsoft, 2021c)

Experimental Instance

To protect the main Visual Studio development environment from potential alterations by untested applications, the Visual Studio SDK offers an experimental space for testing and experimentation. This enables the possibility to develop new applications using the regular Visual Studio interface, but they are run in a separate, isolated Experimental Instance. This ensures that the main development environment remains unaffected. Furthermore, every application equipped with a VSIX package automatically launches the Visual Studio Experimental Instance in debug mode when the application is run.

5.1.2 Commands

A Visual Studio Command provides functionality that can be executed within the IDE. They can range from simple tasks like adding a new file to a project, to more complex tasks like executing test collections. Commands can be triggered in various ways like through menu items, toolbar buttons, keyboard shortcuts or even programmatically. They are meant to impact the general Visual Studio experience by boosting efficiency through automating manual operations.

Definition

Every button in every menu is a command. To add commands to an extension, they need to be defined in the command table.

Listing 1: Custom command group definition

```

1 <Symbols>
2 <!-- This is the package guid. -->
3 <GuidSymbol name="guidCustomExtensionPackage" value="{ad9143c7
   -5915-47e5-a9ac-d46c387072db}" />
4
5 <GuidSymbol name="CustomGroup" value="{119bf355-ef7f-4cdf-af9a-8
   fe6b13c8658}">
6   <IDSymbol name="CSharpProjectMenuGroup" value="0x100" />
7 </GuidSymbol>
8
9 <GuidSymbol name="CommandSet" value="{c97d316c-21f9-4def-9731-4
   d63790b08fd}">
10  <IDSymbol name="CustomCommand" value="0x0100" />
11 </GuidSymbol>
12
13 ...
14 <\Symbols>

```

Listing 1 displays how the symbols are defined for further usage in the command table. These symbols will now be used inside this file with a settable priority tag that determines the ordering of groups or items within the same parent menu or toolbar.

Listing 2: Custom command group definition

```

1 <Groups>
2 <Group guid="CustomGroup" id="CSharpProjectMenuGroup" priority="0
   x003" />
3 </Groups>

```

Listing 2 shows how to use the Group that was defined in Listing 1. This acts as parent element for other components.

Listing 3: Custom command button definition

```

1 <Buttons>
2 <Button guid="CommandSet" id="CustomCommand" priority="0x0100" type=
   "Button">
3   <Parent guid="CustomGroup" id="CSharpProjectMenuGroup" />
4   <Icon guid="guidImages" id="bmpPic1" />
5   <Strings>
6     <ButtonText>Click me</ButtonText>
7   </Strings>
8 </Button>
9 </Buttons>

```

In Listing 3 the required buttons are defined with a parent element, an icon and a text

that is displayed on the button. The icon is defined as a Bitmap, which is not mandatory for this showcase.

Listing 4: Command placement

```
1 <CommandPlacements>
2   <!-- Project Context -->
3   <CommandPlacement guid="CustomGroup" id="CSharpProjectMenuGroup"
4     priority="0x004">
5     <Parent guid="guidSHLMainMenu" id="IDM_VS_CTXT_PROJNODE" />
6   </CommandPlacement>
7 </CommandPlacements>
```

Everything inside the *CustomGroup* is placed at the provided parent element, in this case, the project context menu, as shown in 4.

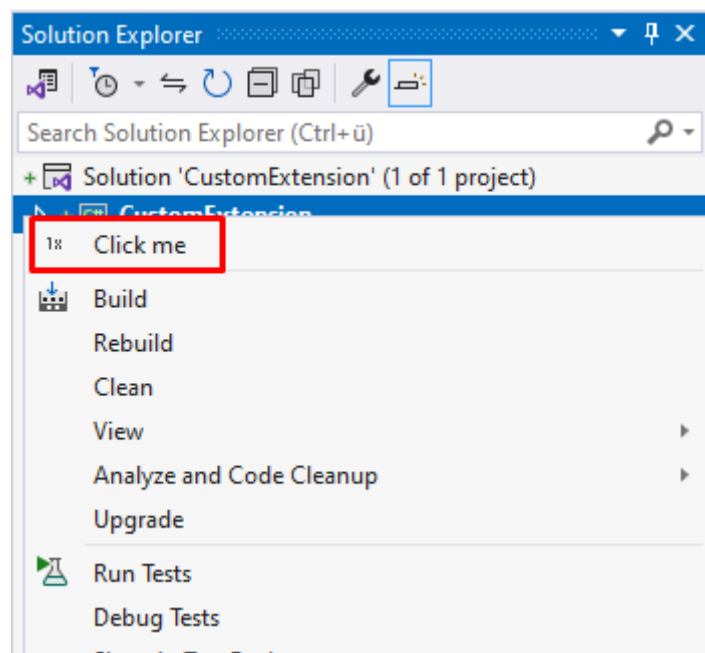


Figure 7: Result of the command definition

After defining the command and coherent elements, the Click-Me button will appear on top of all other buttons when right clicking on a project as shown in Figure 7, considering that the just implemented extension is installed.

Execution

Now that the command is defined in the command table, the invocation logic must be created. Every command is accessed via a GUID and ID from Visual Studio (the ones defined in the command table as displayed in Listing 1). Keep in mind, that if multiple commands are used, creating a base class might be a wise decision.

Listing 5: Possible command base

```

1 public abstract class CommandBase {
2     protected AsyncPackage Package { get; }
3     protected abstract Guid CommandSet { get; }
4     protected abstract int CommandId { get; }
5
6     protected CommandBase(AsyncPackage package, IMenuCommandService
7         commandService) {
8         this.Package = package ?? throw new ArgumentNullException(nameof(
9             package)); ;
10        if (commandService == null) throw new ArgumentNullException(nameof(
11            commandService));
12
13        CommandID menuCommandId = new CommandID(CommandSet, CommandId);
14        OleMenuCommand menuCommand = new OleMenuCommand(this.Execute,
15            menuCommandId);
16
17        commandService.AddCommand(menuCommand);
18    }
19
20    protected abstract void Execute(object sender, EventArgs e);
21    protected IAsyncServiceProvider AsyncServiceProvider => Package;
22    protected IServiceProvider ServiceProvider => Package;
23 }

```

Listing 5 shows a possible base class for commands in Visual Studio. It covers the necessary values that are required by a command that include the *CommandSet* and *CommandId* which are required for the command to be mapped correctly. Code line 11 shows that the *Execute* Method is passed as event handler to the menu command, so it cannot be async without losing the execution state. There exists a well known workaround, for which it also makes sense to create a base class.

Listing 6: Base class for async command handling

```

1 public abstract class AsyncCommandBase : CommandBase {
2     private readonly Action<Exception> onException;
3     private bool isRunning = false;
4
5     protected AsyncCommandBase(AsyncPackage package, IMenuCommandService
6         commandService, Action<Exception> onException) : base(package,
7         commandService) {
8         this.onException = onException;
9     }
10
11    protected override async void Execute(object sender, EventArgs e) {
12        try {
13            if (isRunning)
14                throw new CommandException($"Command {this.GetType().Name} is
15                    already running.");
16
17            isRunning = true;
18            await ExecuteAsync(sender, e);
19        }
20    }
21 }

```

```

16     }
17     catch (Exception ex) {
18         onException?.Invoke(ex);
19     }
20
21     isRunning = false;
22 }
23
24 protected abstract Task ExecuteAsync(object sender, EventArgs e);
25 }

```

Listing 6 shows said workaround. Asynchronous methods with void as return type are a very bad practice, because the execution results in *Fire and Forget*. This workaround provides a Delegate that will be invoked if an exception occurs while command execution.

Listing 7: Custom command

```

1 internal sealed class CustomCommand : AsyncCommandBase {
2     protected override Guid CommandSet => PackageGuids.CommandSet;
3     protected override int CommandId => PackageIds.CustomCommand;
4
5     internal CustomCommand(AsyncPackage package, IMenuCommandService
6         commandService, Action<Exception> onException) : base(package,
7         commandService, onException) {
8         // Initialization work
9     }
10
11     public static CustomCommand Instance { get; private set; }
12     public static async Task InitializeAsync(AsyncPackage package,
13         IMenuCommandService commandService, Action<Exception>
14         onException) {
15         Instance = new CustomCommand(package, commandService, onException)
16         ;
17     }
18
19     protected override async Task ExecuteAsync(object sender, EventArgs
20         e) {
21         // Command Execution
22     }
23 }

```

After creating these classes, the final command class can look like Listing 7 displays. The *PackageGuids* class is generated using the command table, so it provides the required GUIDs and IDs for all defined commands. It appears in certain scenarios, that the *PackageGuids.cs* file is not generated, in this case the *CommandSet* and *CommandId* can simply be created inside the command by copying the values from the command table (in this case the values seen in Listing 1).

Listing 8: Calling command's `InitializeAsync` in package initialization

```
1 protected override async Task InitializeAsync(CancellationTok  
    cancellationToken, IProgress<ServiceProgressData> progress) {  
2     IMenuCommandService commandService = await this.GetServiceAsync(  
        typeof(IMenuCommandService)) as IMenuCommandService;  
3     await CustomCommand.InitializeAsync(this, commandService,  
        OnException);  
4 }
```

It is important to call the Initialization method *InitializeAsync* from inside the package class (*DiscordSupportPackage.cs* from Figure 3) to create the Singleton instance of said command like Listing 8 shows. (Microsoft, 2021c)

5.1.3 Displaying Custom Windows

When working with custom commands, the display of a custom implemented window with i.e. WPF might be required. The *IVsUIShell* provides said functionality.

Listing 9: Displaying a custom window

```
1 public static void Show(System.Windows.Window window) {  
2     // Is required since the Visual Studio Shell should only be used on  
    the main thread  
3     ThreadHelper.ThrowIfNotOnUIThread();  
4  
5     IVsUIShell uiShell = (IVsUIShell)Package.GetGlobalService(typeof(  
        SVsUIShell));  
6     Assumes.Present(uiShell);  
7  
8     uiShell.GetDialogOwnerHwnd(out IntPtr hwnd);  
9     WindowHelper.ShowModal(window, hwnd);  
10 }
```

Listing 10 shows how to access the *IVsUIShell*, which always needs to happen on the main thread, and opening a new custom window using the *WindowHelper* class.

5.1.4 Logging

Besides opening custom windows, it is important to keep the user of the extension updated about what is happening in the background. Especially when operations take a long time. This can happen via creating log files, but a much more suited approach is to create a new *IVsOutputWindowPane* that will then receive the log statements.

Listing 10: Displaying a custom window

```

1 private static IVsOutputWindowPane pane;
2 public static void InitOutputLog(string paneName) {
3     ThreadHelper.ThrowIfNotOnUIThread();
4
5     IVsOutputWindow logWindow = AsyncPackage.GetGlobalService(typeof(
6         SVsOutputWindow)) as IVsOutputWindow;
7     Guid guid = Guid.NewGuid();
8     logWindow.CreatePane(ref guid, paneName, 1, 1);
9     logWindow.GetPane(ref guid, out pane);
10 }
11
12 public static void OutputWindowFunc(LogStatement obj) {
13     Func<Task> logTask = new Func<Task>(async () => {
14         await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();
15         pane.OutputStringThreadSafe(obj.ToString() + "\n");
16     });
17     ThreadHelper.JoinableTaskFactory.Run(logTask);
18 }

```

Listing 10 shows how the pane is created using the *IVsOutputWindow* and can then be accessed by loggers to provide logs to the created output window pane. It is also important to not invoke tasks by hand, but let the *ThreadHelper.JoinableTaskFactory* handle task execution to avoid deadlocks.

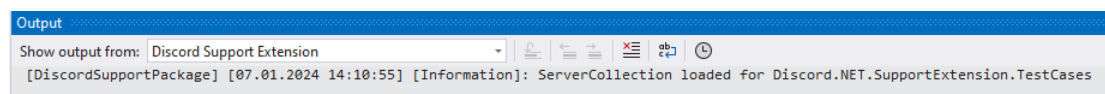


Figure 8: Custom output window pane

For instance Figure 8 shows that there exists a new *IVsOutputWindowPane* called "Discord Support Extension" that provides context specific information.

5.2 ~~MS~~ Roslyn

Microsoft's Roslyn, also known as the .NET Compiler Platform SDK, represents a significant change in how .NET handles compilation and provides extensive APIs for various compilation phases. Roslyn is essentially a set of services built around two key principles: Extensibility and Maintainability. This design allows for layers above and integration with third party plugins, offering well documented and supported public APIs. External developers can leverage these services to do a variety of innovative things. Some of these can be crafting their own tools tailored to any specific layer of programming tasks or developing simple plugins like diagnostic analyzers, code fixes, refactorings, and IntelliSense providers for certain layers (Vasani, 2017). Roslyn plays an essential role to achieve the wanted goal with the Visual Studio Extension, because it enables the analysis of a users code during the coding process. This is required because the completions pushed to the Editor API are context dependent, so it is important that the context is determined live while the user is coding.

5.2.1 Compiler phases

Roslyn deconstructs the compiler into distinct phases and exposes managed APIs for each phase, making language compilers a service (Developer.com, 2012). The .NET Compiler Platform SDK is a set of tools that simplifies building C# and VB compilers, following steps from analyzing text to creating executable code. It provides tools for each step to work with the code being compiled (Microsoft, 2021b).

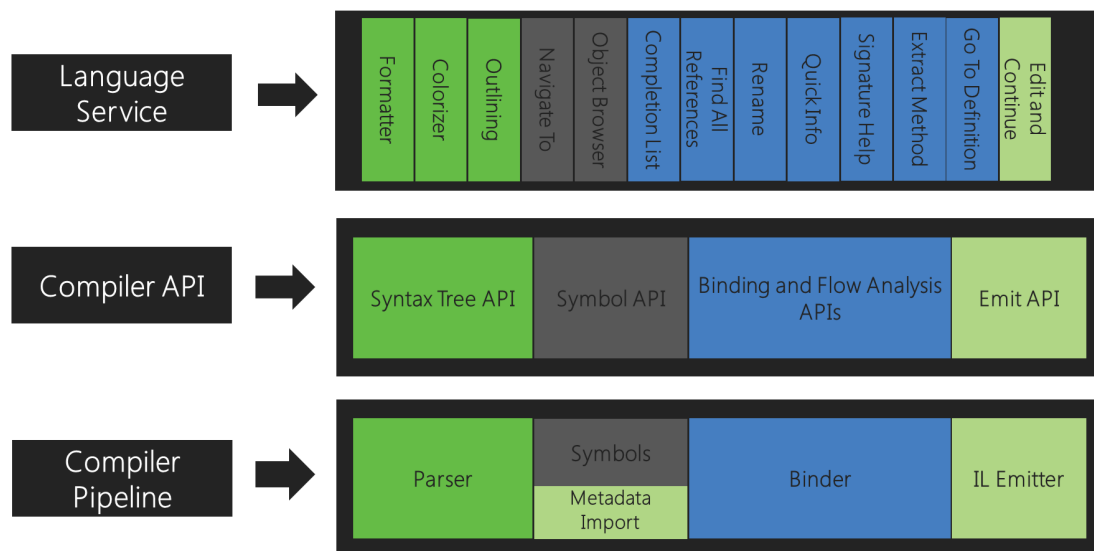


Figure 9: Compiler pipeline functional areas¹

Figure 9 indicates that upon the Compiler Pipeline the Compiler API and Language Service are built. To analyse ones code, the Compiler API is required to access syntax trees and the Symbol API which contains the semantic information.

¹Source: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/compiler-api-model/compiler-pipeline-lang-svc.png> (2024-01-07)

5.2.2 Code Analysis

Code analysis in general is a big topic, but in this scenario, where the users code is analysed after the IntelliSense process is triggered, there are only a few key concepts required. These concepts consist of Workspaces, Syntax and Semantics, so a core understanding of how these things work is required.

Workspaces and Solutions

Workspaces are essential for code analysis and refactoring across entire solutions. It organizes project information into a unified object model, allowing easy access to important compiler elements like syntax trees and semantic models, without file parsing or dependency management. Within an IDE, workspaces reflect real-time changes, updating the model as users proceed to make updates like writing code or changing properties (Microsoft, 2021d). Even though workspaces can change with each key press, the solution model can be worked with independently. Solutions are immutable models containing projects and documents, allowing for sharing without conflicts or duplication. Once a solution instance is obtained from the `Workspace.CurrentSolution` property, it remains unaltered. Modifications to solutions involve creating new instances based on existing ones and applying these changes back to the workspace. Projects within the solution model contain source code documents, parsing and compilation settings, and references, facilitating direct access to compilations. Documents in the model represent individual source files, from which text, syntax tree, and semantic model can be accessed. (Microsoft, 2021d)

Listing 11: Roslyn Workspace

```
1 IComponentModel componentModel = (IComponentModel)Package.  
    GetGlobalService(typeof(SComponentModel));  
2 VisualStudioWorkspace workspace = componentModel.GetService<Microsoft.  
    VisualStudio.LanguageServices.VisualStudioWorkspace>();  
3  
4 Solution solution = workspace.CurrentSolution;  
5  
6 foreach (Project project in solution.Projects) {  
7     Console.WriteLine($"Project: {project.Name}");  
8  
9     foreach (Document document in project.Documents) {  
10        Console.WriteLine($"\\tDocument: {document.Name}");  
11    }  
12 }
```

Listing 11 shows how to access Visual Studio's workspace and getting a snapshot of the current solution that is open inside the workspace.

Syntax

Syntax Trees provide a detailed, hierarchical representation of source code's structure,

essential for analyzing and manipulating code in tools like IDEs. They are immutable, ensuring thread safety and full fidelity, meaning they represent every piece of source text, including whitespaces and comments. Syntax Nodes within these trees represent the syntactic constructs like expressions or statements. They are interconnected which means they are forming the tree's structure. Syntax Tokens are the smallest elements, representing keywords, identifiers, and punctuation, playing a key role in the tree's construction and the interpretation of the code's syntax. Syntax Trivia covers more unimportant text like spaces or comments. Spans indicate the text range of nodes or tokens. Syntax Kinds are used to categorize syntax elements, supporting their identification (Microsoft, 2021f).

Example

Listing 12: Syntax Example

```
1 SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
2 public class CustomClass {
3     public void CustomMethod() {
4     }
5 }");
6
7 CompilationUnitSyntax root = tree.GetRoot() as CompilationUnitSyntax;
8
9 foreach (ClassDeclarationSyntax classDeclaration in root.
    DescendantNodes().OfType<ClassDeclarationSyntax>()) {
10     Console.WriteLine($"Class: {classDeclaration.Identifier.ValueText}")
11     ;
12     foreach (MethodDeclarationSyntax methodDeclaration in
        classDeclaration.DescendantNodes().OfType<
            MethodDeclarationSyntax>()) {
13         Console.WriteLine($"Method: {methodDeclaration.Identifier.
            ValueText}");
14     }
15 }
```

Listing 12 shows a string representing valid C# syntax that is used to create a *SyntaxTree*. Then the root is accessed to iterate through the descendant nodes to identify classes and methods, which identifier values are printed to the console.

Listing 13: Syntax Example Output

```
1 Class: CustomClass
2 Method: CustomMethod
```

Listing 13 shows the output of the executed Listing 12.

Compilation and Symbols

The Semantic model in compilation extends beyond syntax, interpreting elements like types and variables, essential for understanding code without direct access to its source, such as in libraries. It represents a deep layer of analysis, connecting names in code to their meanings and behaviors through symbols, each signifying a unique element like a class or method, derived from compiler insights or imported metadata. Symbols and the Semantic model provide a comprehensive framework for navigating and interpreting code's logical structure, essential for tasks like type checking and resolving references within a compilation's immutable context (Microsoft, 2021e).

Example

Listing 14: Semantics Example

```
1 SyntaxTree tree = CSharpSyntaxTree.ParseText(@"
2 public class CustomClass {
3     public void CustomMethod(int customParameter) {
4     }
5 }");
6
7 SyntaxNode root = tree.GetRoot();
8 Compilation compilation = CSharpCompilation.Create("CustomCompilation"
9 )
10 .AddReferences(MetadataReference.CreateFromFile(typeof(object).
11     Assembly.Location))
12 .AddSyntaxTrees(tree);
13
14 SemanticModel model = compilation.GetSemanticModel(tree);
15 MethodDeclarationSyntax methodDeclaration = root.DescendantNodes().
16     OfType<MethodDeclarationSyntax>().First();
17 IMethodSymbol symbol = model.GetDeclaredSymbol(methodDeclaration);
18
19 Console.WriteLine($"Method: {symbol.Name}");
20 foreach (IParameterSymbol parameter in symbol.Parameters) {
21     Console.WriteLine($"Parameter: {parameter.Name}, Type: {parameter.
22         Type}");
23 }
```

Listing 14 shows a string representing valid C# syntax that is used to create a *SyntaxTree* and builds the compilation based on that tree. Then the compilation is used to acquire a *SemanticModel*, that is then used to get the symbol from the first found *MethodDeclarationSyntax*. All parameters are iterated and its name and type are printed to the console.

Listing 15: Semantics Example Output

```

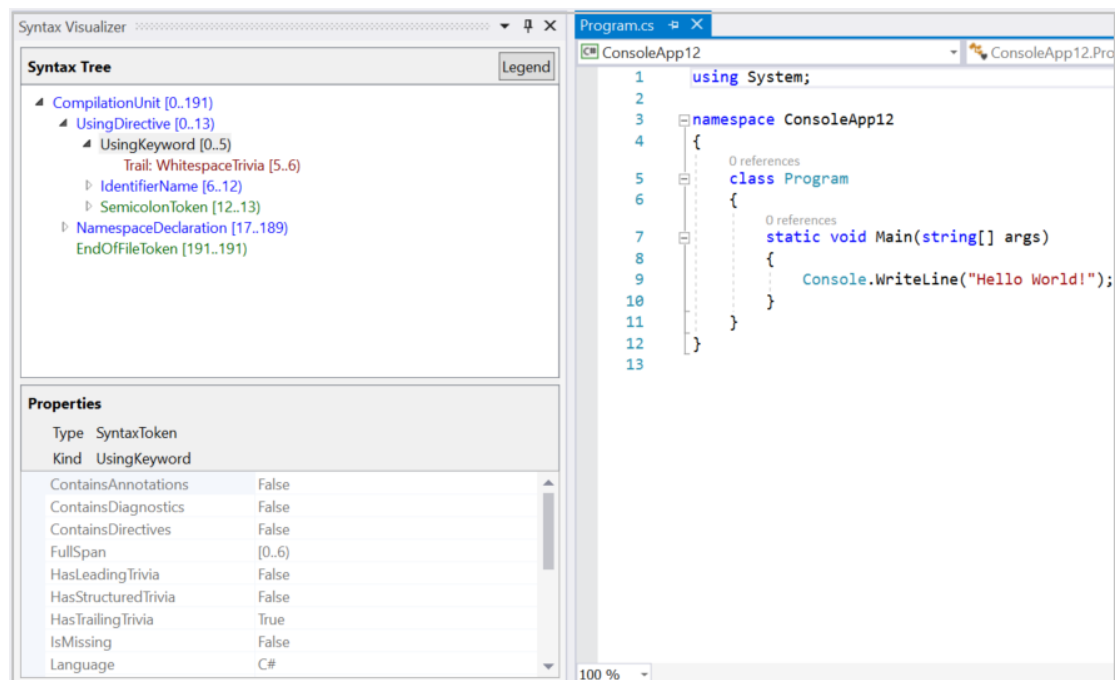
1 Method: CustomMethod
2 Parameter: customParameter, Type: System.Int32

```

Listing 15 shows the output of the executed Listing 14.

5.2.3 Syntax Visualizer in Visual Studio

“The Syntax Visualizer enables inspection of the syntax tree for the C# or Visual Basic code file in the current active editor window inside the Visual Studio IDE. The visualizer can be launched by clicking on View > Other Windows > Syntax Visualizer. You can also use the Quick Launch toolbar in the upper right corner. Type "syntax", and the command to open the Syntax Visualizer should appear“ (Microsoft, 2022)

Figure 10: Syntax Visualizer²

²Source: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/media/syntax-visualizer/visualize-csharp.png> (2024-01-07)

5.3 Editor API

The Microsoft Visual Studio Editor API contains open source layers of the Visual Studio editor, including public API definitions and implementations for text model, logic and editor operations. It is designed for extensibility creators to integrate with Visual Studio (Microsoft, 2021a).

5.3.1 Async Completion API

The Async Completion API is a subcomponent of the Editor API focusing on asynchronous operations to enhance IntelliSense features. The most important one for this use case being the addition of completion items using the *IAsyncCompletionSourceProvider* with an *IAsyncCompletionSource* as well as handling how these items will be committed to the text buffer utilizing the *IAsyncCompletionCommitManagerProvider* with an *IAsyncCompletionCommitManager*. Visual Studio uses the Managed Extensibility Framework to discover and instantiate components that inherit from interfaces like *IAsyncCompletionSourceProvider* and *IAsyncCompletionCommitManagerProvider*. MEF scans the assembly metadata for exported components marked with specific attributes indicating their purpose and capabilities. When Visual Studio requires these services, MEF composes the parts together, linking the providers to the editor based on the exported metadata. This mechanism allows for a decoupled, extensible architecture where extensions can be developed and integrated seamlessly into the Visual Studio environment, enhancing the editing and coding experience without direct modification to the core IDE (Wieczorek, 2018a).

5.4 Managed Extensibility Framework

The Managed Extensibility Framework (MEF) is a tool designed to build extensible and lightweight applications. Since MEF is used by Visual Studio like mentioned in Subsection 5.3.1, a basic understanding of how exporting with MEF works is required. It is basically a ~~Dependency-Injection~~ ^(DI) container but differs from other containers in its focus on extensibility and part discovery. Unlike traditional DI containers designed mainly for dependency management within an application, MEF aims to make applications extendable by allowing parts to be added or removed without requiring changes to the core application. It uses a catalog and composition model to discover and compose parts at runtime, which makes it ideal for applications that require incorporate extensions or plugins.

gut, dass sie das ergänzt haben

Composition container

At the heart of the MEF composition model lies the composition container, serving as the repository for all available parts and leading their composition. Composition is the matching up of imports to exports.

Imports and Exports

Listing 16: ICustomHandler for MEF example

```
1 public interface ICustomHandler {  
2     string Handle(string input);  
3 }
```

Listing 16 shows the design of an interface that is required for later usage.

Listing 17: CustomHandler ~~Export~~ for MEF example

```
1 [Export(typeof(ICustomHandler))]  
2 class CustomHandler : ICustomHandler {  
3 }
```

Listing 17 shows how the Export attribute is used. For a successful match between an import and an export, both must share the same contract. If an export is associated with a different contract, such as `typeof(CustomHandler)`, it won't match an import looking for a different contract, leading to the import not being fulfilled. The contract must be an exact match.

Listing 18: *ICustomHandler* Import for MEF example

```
1 [Import(typeof(ICustomHandler))]  
2 public ICustomHandler customHandler;
```

Listing 18 displays how to import an exported component. In MEF, every import is associated with a contract that determines which exports it matches with. This contract can be an explicit string or automatically generated based on a type, like the *ICustomHandler* interface in this context. Any export that declares a matching contract can fulfill this import. The contract is not necessarily tied to the type of the importing object. Even though the object type might be *ICustomHandler*, the contract can differ. By default, MEF uses the type of the import to determine the contract, unless explicitly specified otherwise.

Gesamtüberblick zu Kapitel 6

- Dieses Kapitel enthält viele Implementierungsdetails, die man nur verstehen kann, wenn man sich intensiv mit der Materie beschäftigt. Es soll mir ein wenig die Führung des Lesers, damit er ihre Ausführungen besser einordnen kann.
- Überlegen Sie, ob Sie aus Abschnitt 6.4 (Resultate) nicht ein eigenes Kapitel machen.



6 BUILDING THE VISUAL STUDIO EXTENSION

6 Building the Visual Studio Extension

Since there will only be small code snippets and explanations, the whole project is available on Github: <https://github.com/TheHandsomeBanana/Discord.NET.SupportExtension>.

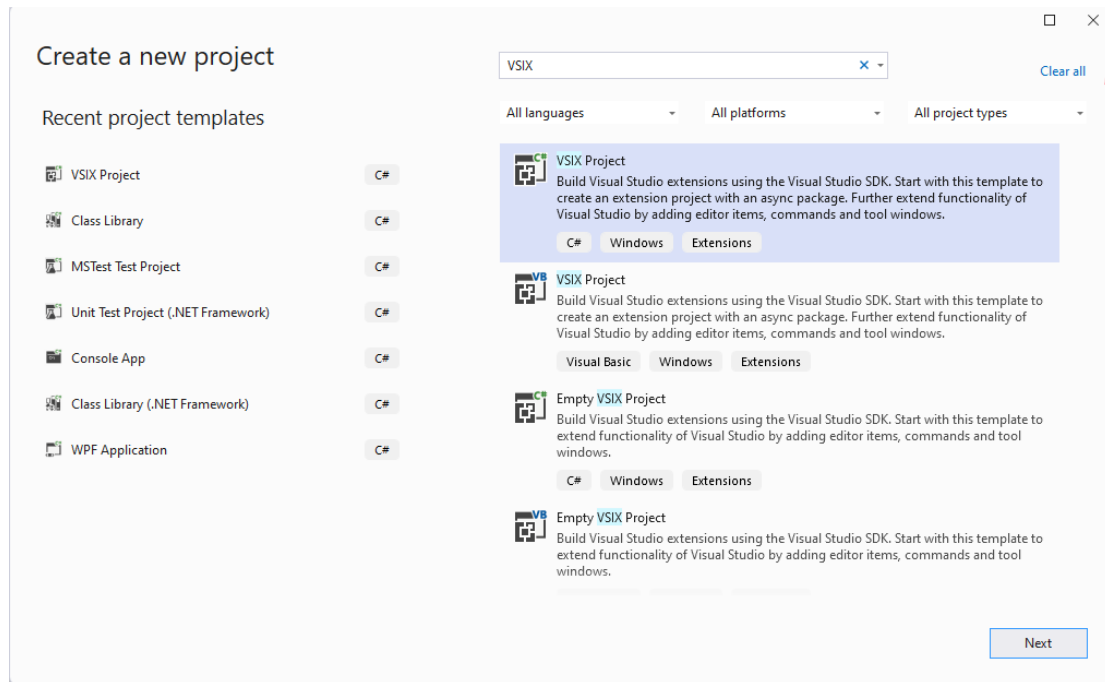


Figure 11: References of the newly created Visual Studio Project

To create a new Visual Studio Extension, it is not required to start from scratch, there already exists a template as Figure 11 shows.

- Das Literaturverzeichnis müssen Sie unbedingt noch aufbereiten (siehe diesbezügliche Anmerkungen).

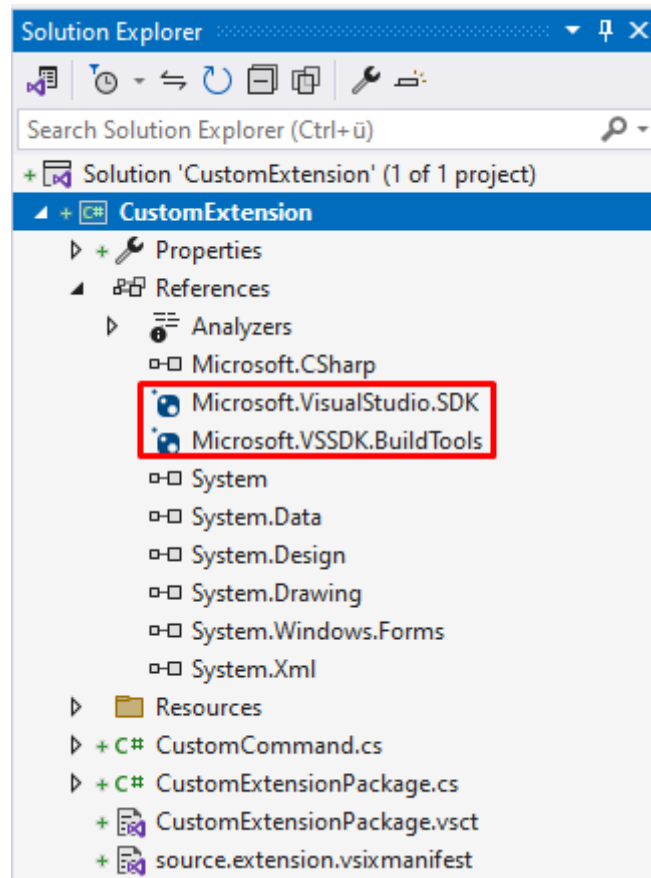


Figure 12: References of the newly created Visual Studio Project

The necessary NuGet packages for extension development are already referenced by default in this template as Figure 12 ^{shows} displays.

6.1 VS Commands

As already mentioned in Section 5 there are 3 commands required for this use-case.

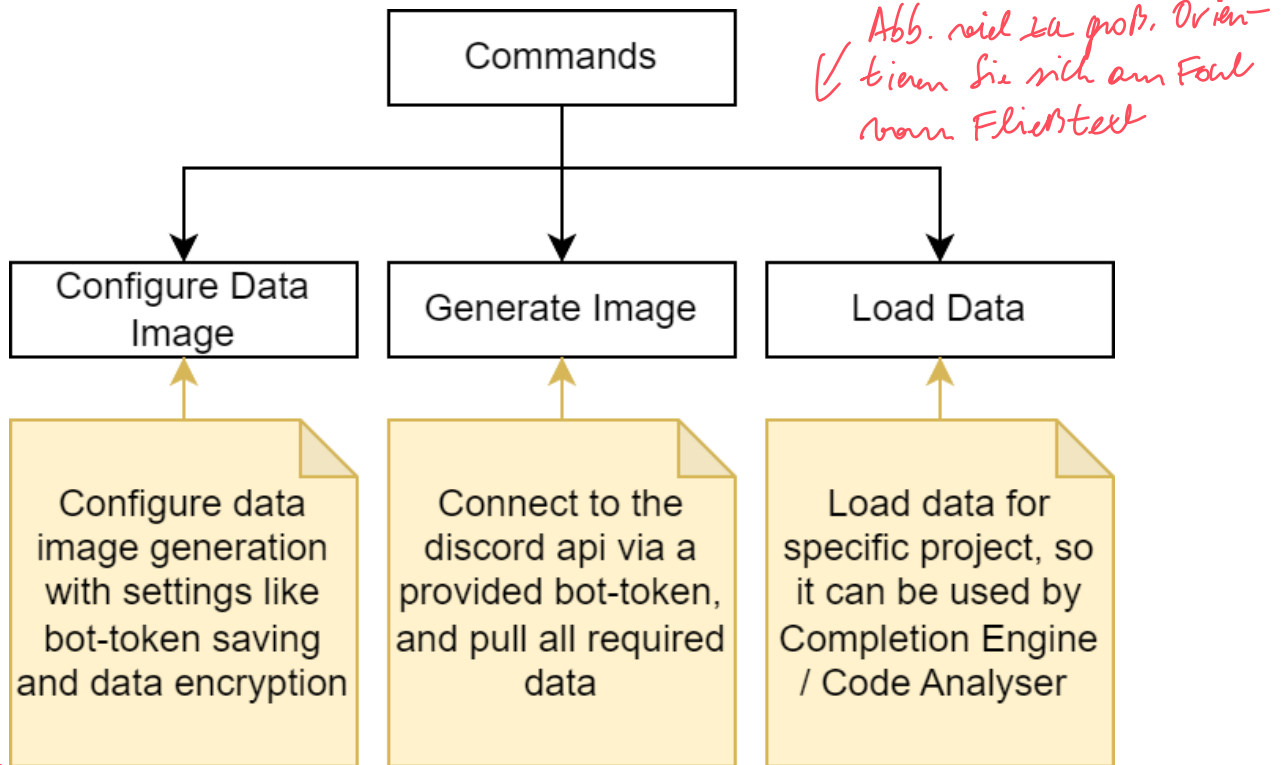


Figure 13: Workflow for commands

geben Sie das in den Fließtext.

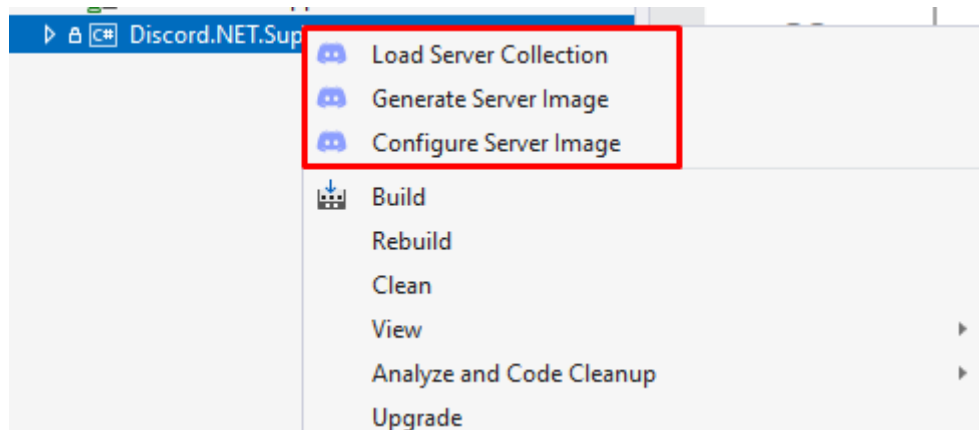


Figure 14: Commands in project context

Figure 14 and 13 shows the 3 commands with its functionality and how they can be used when right-clicking on a project.

Configure Server Image

Since the data for the IntelliSense process should ^{be} cached to keep performance up to track, a way to get this data from the API by command is required. To acquire said functionality, a custom window is implemented that takes the users bot token and additional settings like encryption methods, which can then be used to download all servers data that this bot is part of.

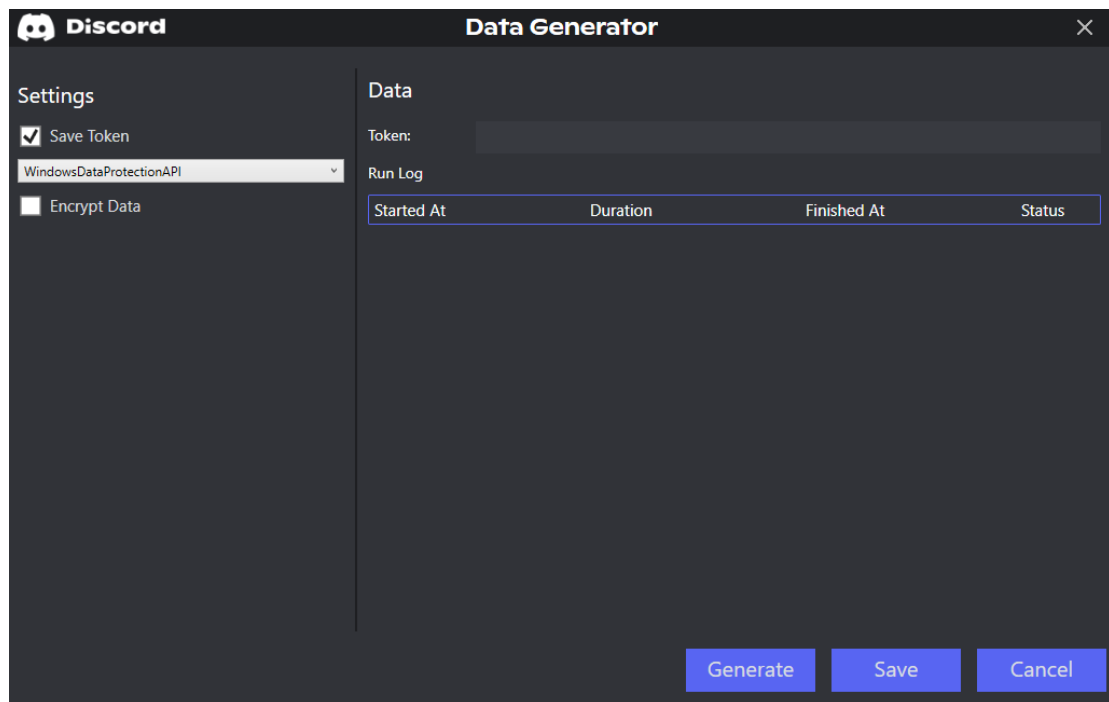


Figure 15: Command that configures how the data will be generated

Figure 15 shows the custom window that is used to configure the data generation. ^{process} It is possible to save the token, which will then be ^{stored} saved using the selected cryptography mode. Supported ways to encrypt and decrypt the token are the Windows Data protection API (DPAPI) and AES. Same is possible for the downloaded data. When clicking *Save*, the configurations data is saved as JSON and added to the project as a new file called *DiscordServerImageConfig.json*.

Listing 19: DiscordServerImageConfig.json

```

1 {
2   "SaveToken": true,
3   "EncryptData": false,
4   "TokenEncryptionMode": null,
5   "DataEncryptionMode": null,
6   "TokenKeyIdentifier": "",
7   "DataKeyIdentifier": null,
8   "Token": "",
9   "RunLog": []

```

```
10 }
```

Listing 19 shows how the configuration looks like in JSON format. When extending this whole process, it definitely makes sense to use a database instead of storing all information, especially the run-log inside the configuration file.

Generate Server Image

To generate the server image, the bot token is used to connect to Discord's API and load the required data.

Listing 20: Generating the data loaded from Discord's API

```
1 IDiscordEntityService entityService = container.Resolve<
  IDiscordEntityService>(nameof(DiscordRestEntityService));
2 try {
3     entityService.OnTimeout += OnTimeout;
4
5     InteractionHelper.MapDataEncryptToEntityService(entityService, model
      , logger, out bool cancel);
6     if (cancel) {
7         logger.LogInformation(InteractionMessages.GenerationAborted);
8         return;
9     }
10
11     await entityService.Connect(token);
12     if (entityService.Ready) {
13         DiscordServerCollection serverCollection = await entityService.
            LoadEntities();
14
15         await entityService.SaveToFile(currentCachePath, serverCollection)
            ;
16         logger.LogInformation(InteractionMessages.GenerateImageSuccess);
17         UIHelper.ShowInfo(InteractionMessages.GenerateImageSuccess);
18
19         CommandHelper.RunVSCommand(CommandSet, PackageIds.
            LoadServerCollectionCommand);
20
21         status = JobStatus.Succeeded;
22     }
23     await entityService.Disconnect();
24 }
25 finally {
26     await entityService.DisposeAsync();
27 }
```

Listing 20 represents a snippet of the Generate Server Image command, where the *IDiscordEntityService* is used to download the required data using the provided bot token. This service is a custom implementation utilizing Discord.Net's functionality to connect

^{the} to Discord's API and access servers, channels, users and so on through a client that is either REST or Socket based.

Listing 21: Implementation of the `DiscordRestEntityService`

```

1 public class DiscordRestEntityService : DiscordBaseEntityService,
   IDiscordEntityService {
2
3     public DiscordRestEntityService(ILoggerFactory loggerFactory,
   IAsyncStreamHandler streamHandler) : base(loggerFactory,
   streamHandler) {
4         Client = new DiscordRestClient(new DiscordRestConfig() {
5             DefaultRetryMode = RetryMode.AlwaysRetry
6         });
7     }
8
9     ...
10
11    public async Task<DiscordServerCollection> LoadEntities() {
12        ...
13
14        IEnumerable<DiscordServer> servers = await Task.WhenAll((await ((
15            DiscordRestClient)Client).GetGuildsAsync())
16            .Select(async e => new DiscordServer() {
17                Id = e.Id,
18                Name = e.Name,
19                ParentId = null,
20                Type = DiscordEntityType.Server,
21                UserCollection = await GetUsers(e),
22                ChannelCollection = await GetChannels(e),
23                RoleCollection = await GetRoles(e)
24            }));
25
26        Logger.LogInformation($"{servers.Count()} servers data downloaded
27            from Discord API.");
28        return new DiscordServerCollection(servers);
29    }
30    ...
31 }

```

Listing 21 shows a snippet of the REST-based `IDiscordEntityService` where a new `DiscordRestClient` provided by `Discord.Net` is created and then used to access all servers asynchronously (`GetGuildsAsync()`). The data is then mapped to custom models (`DiscordServer`, `DiscordUser`, ...) that contain only relevant data for the completion items. The generated data is saved to a file inside the `AppData/Local` folder.

Listing 22: Generated data

```

1 {
2   "1157751183184760953": {
3     "Type": 0,
4     "UserCollection": {
5       "270995861826306049": {
6         "Type": 1,
7         "Id": 270995861826306049,
8         "Name": ".handsomebanana",
9         "ParentId": 1157751183184760953
10      },
11      "948672754272583721": {
12        "Type": 1,
13        "Id": 948672754272583721,
14        "Name": "Main Banana Man",
15        "ParentId": 1157751183184760953
16      }
17    },
18    "RoleCollection": {
19      "1157751183184760953": {
20        "Type": 2,
21        "Id": 1157751183184760953,
22        "Name": "@everyone",
23        "ParentId": 1157751183184760953
24      }
25    },
26    "ChannelCollection": {
27      "1157751183822311494": {
28        "Type": 3,
29        "ChannelType": 2,
30        "Id": 1157751183822311494,
31        "Name": "Textkanale",
32        "ParentId": 1157751183184760953
33      },
34      "1157751183822311495": {
35        "Type": 3,
36        "ChannelType": 2,
37        "Id": 1157751183822311495,
38        "Name": "Sprachkanale",
39        "ParentId": 1157751183184760953
40      }
41    },
42    "Id": 1157751183184760953,
43    "Name": "Discord.NET Support Extension Testserver",
44    "ParentId": null
45  }
46 }

```

Listing 22 shows how the generated data looks like. As seen in this example the bot used to generate this data is called *Main Banana Man* which is also part of the user collection.

Load Server Collection

The data structure to hold the data in memory dynamically loads it per project. On startup the currently selected or active project is used if the required data exists. For every other project, the *Load Server Collection* command comes in handy, to dynamically load the required data used for completion items.

6.2 MEF Components Project

This project contains the MEF components, which utilizes functionality from the Async Completion Api mentioned in Subsection 5.3.1.

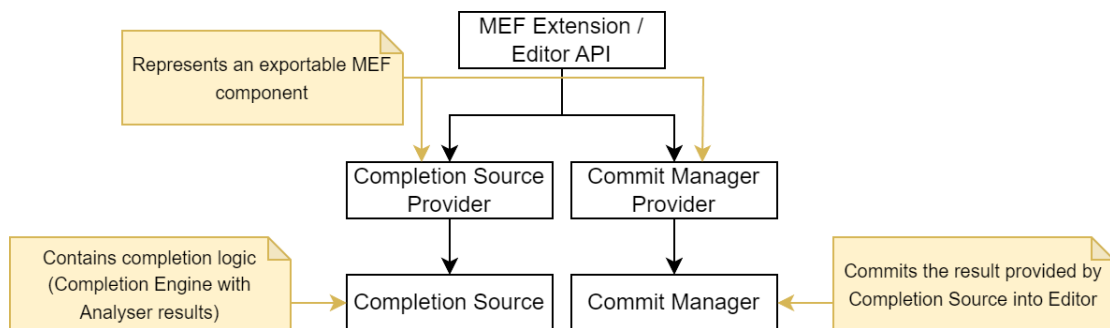


Figure 16: Workflow for the MEF Project

Figure 16 shows the basic structure, the *MEF Extension* resembling the project containing the MEF components, which include the *IAsyncCompletionSourceProvider* and *IAsyncCompletionCommitManagerProvider*.

Completion Source

Listing 23: Discord Completion Source Provider

```

1 [Export(typeof(IAsyncCompletionSourceProvider))]
2 [ContentType("CSharp")]
3 [Name("Discord Source Provider")]
4 public class AsyncDiscordCompletionSourceProvider :
    IAsyncCompletionSourceProvider {
5     public IAsyncCompletionSource GetOrCreate(ITextView textView) {
6         return new AsyncDiscordCompletionSource();
7     }
}

```

Listing 23 shows that classes inheriting from *IAsyncCompletionSourceProvider* implement the method *GetOrCreate(ITextView textView)*, that returns an *IAsyncCompletionSource*. That return value contains the added completion items that are pushed into the editor (which is done by Visual Studio). “When the user interacts with Visual Studio,

e.g. by typing, the editor will see if it is appropriate to begin a completion session“ (Wieczorek, 2018b).

Listing 24: Discord Completion Source

```

1 public async Task<CompletionContext> GetCompletionContextAsync(
    IAsyncCompletionSession session, CompletionTrigger trigger,
    SnapshotPoint triggerLocation, SnapshotSpan applicableToSpan,
    CancellationToken token) {
2     ...
3
4     // This engine is a custom implementation that performs code
        analysis and is part of a DI container from which the instance
        is resolved.
5     IDiscordCompletionEngine engine = supportPackageContainer.Resolve<
        IDiscordCompletionEngine>();
6
7     try {
8         ...
9         // The documentIdentifier is set in InitializeCompletion(...)
10        Microsoft.CodeAnalysis.Document document = vsWorkspace.
            CurrentSolution.GetDocument(documentIdentifier);
11
12        ...
13
14        SemanticModel semanticModel = await document.GetSemanticModelAsync
            (token);
15        SyntaxToken triggerToken = (await document.GetSyntaxRootAsync(
            token)).FindToken(triggerLocation);
16
17        // The engine starts the required code analysers that will build
            completions based on the analysed code
18        DiscordCompletionItem[] completions = await engine.
            ProcessCompletionAsync(vsWorkspace.CurrentSolution,
            semanticModel, triggerToken);
19
20        ...
21
22        return new CompletionContext(completions.Select(e => {
23            CompletionItem ci = new CompletionItem(...);
24            ci.Properties.AddProperty("description", e.Description); // The
                discription called in GetDescriptionAsync(..)
25            return ci;
26        }).ToImmutableArray());
27    }
28    catch (Exception ex) {
29        ...
30    }
31
32    return default;
33 }
34
35 public async Task<object> GetDescriptionAsync(IAsyncCompletionSession
    session, CompletionItem item, CancellationToken token) {
36    // The discription of an item displayed in the IntelliSense
        completion list

```

```

37 }
38
39 public CompletionStartData InitializeCompletion(CompletionTrigger
    trigger, SnapshotPoint triggerLocation, CancellationToken token) {
40     // Initialization work i.e. setting the current document that is
        analysed in GetCompletionContextAsync(..)
41     // Also defines if this completion source (this class) participates
        in completion (which means GetCompletionContext(..) will be
        called)
42 }

```

In Listing 23 a new *AsyncDiscordCompletionSource* is created. The API will then proceed to call the methods defined in the *IAsyncCompletionSource* interface if IntelliSense is triggered. “We will attempt to get completion items by asynchronously calling *GetCompletionContextAsync* where you provide completion items” (Wieczorek, 2018b). As shown in Listing 24 the methods *GetCompletionContextAsync*, *GetDescriptionAsync* and *InitializeCompletion* are implemented based on this use case and are explained each as comment in the Listing.

↑
gehören diese zur Klasse AsyncDiscordCompletionSource?

Completion Commit Manager

Listing 25: Discord Completion Commit Manager Provider

```

1 [Export(typeof(IAsyncCompletionCommitManagerProvider))]
2 [ContentType("CSharp")]
3 [TextViewRole(PredefinedTextViewRoles.Editable)]
4 [Order(Before = "default")]
5 [Name("Discord Commit Provider")]
6 public class AsyncDiscordCompletionCommitManagerProvider :
    IAsyncCompletionCommitManagerProvider {
7     public IAsyncCompletionCommitManager GetOrCreate(ITextView textView)
        {
8         return new AsyncDiscordCompletionCommitManager();
9     }
10 }

```

Similar as in Listing 23, Listing 25 shows that classes inheriting from *IAsyncCompletionCommitManagerProvider* implement the method *GetOrCreate(ITextView textView)*, that returns an *IAsyncCompletionCommitManager*. That return value contains the logic responsible for committing completion items into the editor. “We use this interface to determine under which circumstances to commit (insert the completion text into the text buffer and close the completion UI) a completion item.” (Wieczorek, 2018b)

Listing 26: Discord Completion Commit Manager

```

1 public class AsyncDiscordCompletionCommitManager :
    IAsyncCompletionCommitManager {
2     public IEnumerable<char> PotentialCommitCharacters => ". ".
        ToCharArray();
3
4     public bool ShouldCommitCompletion(IAsyncCompletionSession session,
        SnapshotPoint location, char typedChar, CancellationToken token)
    {
5         return false; // No custom commit characters should commit the
            completion item into the editor
6     }
7
8     public CommitResult TryCommit(IAsyncCompletionSession session,
        ITextBuffer buffer, CompletionItem item, char typedChar,
        CancellationToken token) {
9         // If the item is from our source, insert the currently selected
            completion item with all required information into the
            textbuffer
10        if (item.Source is AsyncDiscordCompletionSource) {
11            buffer.Replace(session.ApplicableToSpan.GetSpan(buffer.
                CurrentSnapshot), item.InsertText + $" /* {item.DisplayText}
                ({item.Suffix}) */");
12            return CommitResult.Handled;
13        }
14
15        return CommitResult.Unhandled;
16    }
17 }

```

In Listing 25 a new *AsyncDiscordCompletionCommitManager* is created. This class is responsible for managing how the completion items added by the *AsyncDiscordCompletionSource* (from Listing 24) are committed into the editor. “When we first create the completion session, we access the *PotentialCommitCharacters* property that returns characters that potentially commit completion when user types them. We access this property, therefore it should return a preallocated array. Typically, the commit characters include space and other token delimiters such as ., (,). Don’t worry about Tab and Enter, as they are handled separately. If a character is a commit character in some, but not all situations, you must add it to this collection. Characters from available *IAsyncCompletionCommitManagers* are combined into a single collection for the duration of the completion session.” (Wieczorek, 2018a)

Dr. Kürzel

6.3 Code Analyser

In Listing 24 the current document is available through working with the Development Tools Environment (DTE). The DTE is used to access many different kinds of features within Visual Studio. Therefore the currently open document id is accessible, which is then utilized to access the current document. This information is crucial to get the semantic model, which is also mandatory for analysing the users code. The method *GetCompletionContextAsync* from Listing 24 provides a *SnapshotPoint* as parameter. With a *SnapshotPoint* and the available document, the *SyntaxToken*, where IntelliSense

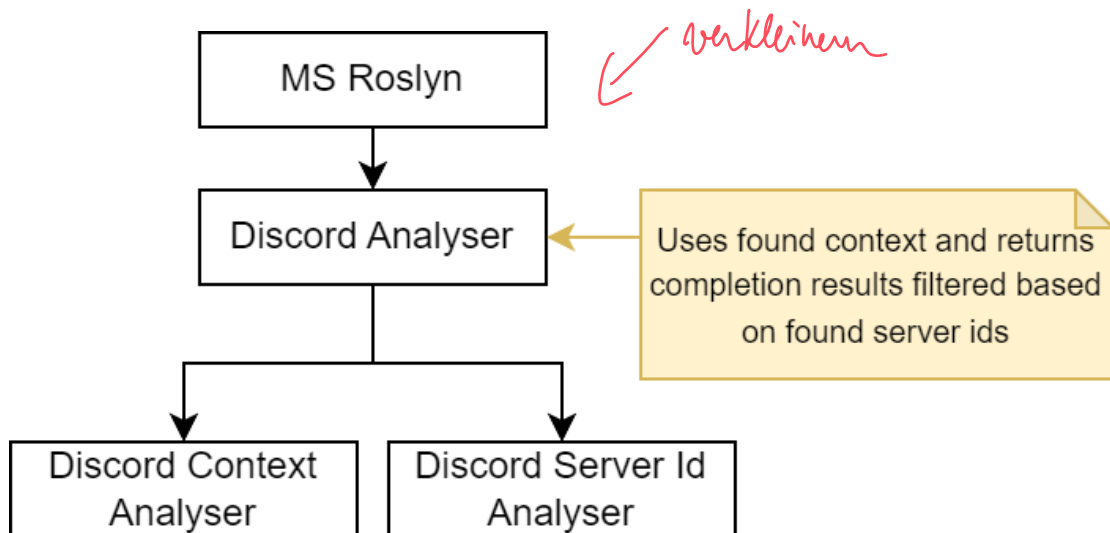
has been triggered, can be acquired.

Listing 27: Getting the semantic model and syntax token

```
1 SemanticModel semanticModel = await document.GetSemanticModelAsync(
    token);
2 SyntaxToken triggerToken = (await document.GetSyntaxRootAsync(token)).
    FindToken(triggerLocation);
```

Listing 27 shows the above mentioned logic. The *document* resembles the file that is currently open in the editor and the *triggerLocation* contains information about where in the currently open file IntelliSense was triggered. The *token* resembles a *Cancellation-Token*, it is a good practice to use cancellation tokens when using asynchronous code. The semantic model and syntax token are then used to get the necessary information about the users current state of code.

Sätze ver-
binden.



verkleinern

Figure 17: Workflow for the implemented code analyser using Roslyn

Figure 17 displays a structure of 3 analysers. The main analyser that collects information through 2 sub analysers. The main analyser is called by the *IDiscordCompletionEngine*, the use of which is shown at Listing 24.

The Discord Analyser

Discord Context Analyser and ...

Discord Analyser

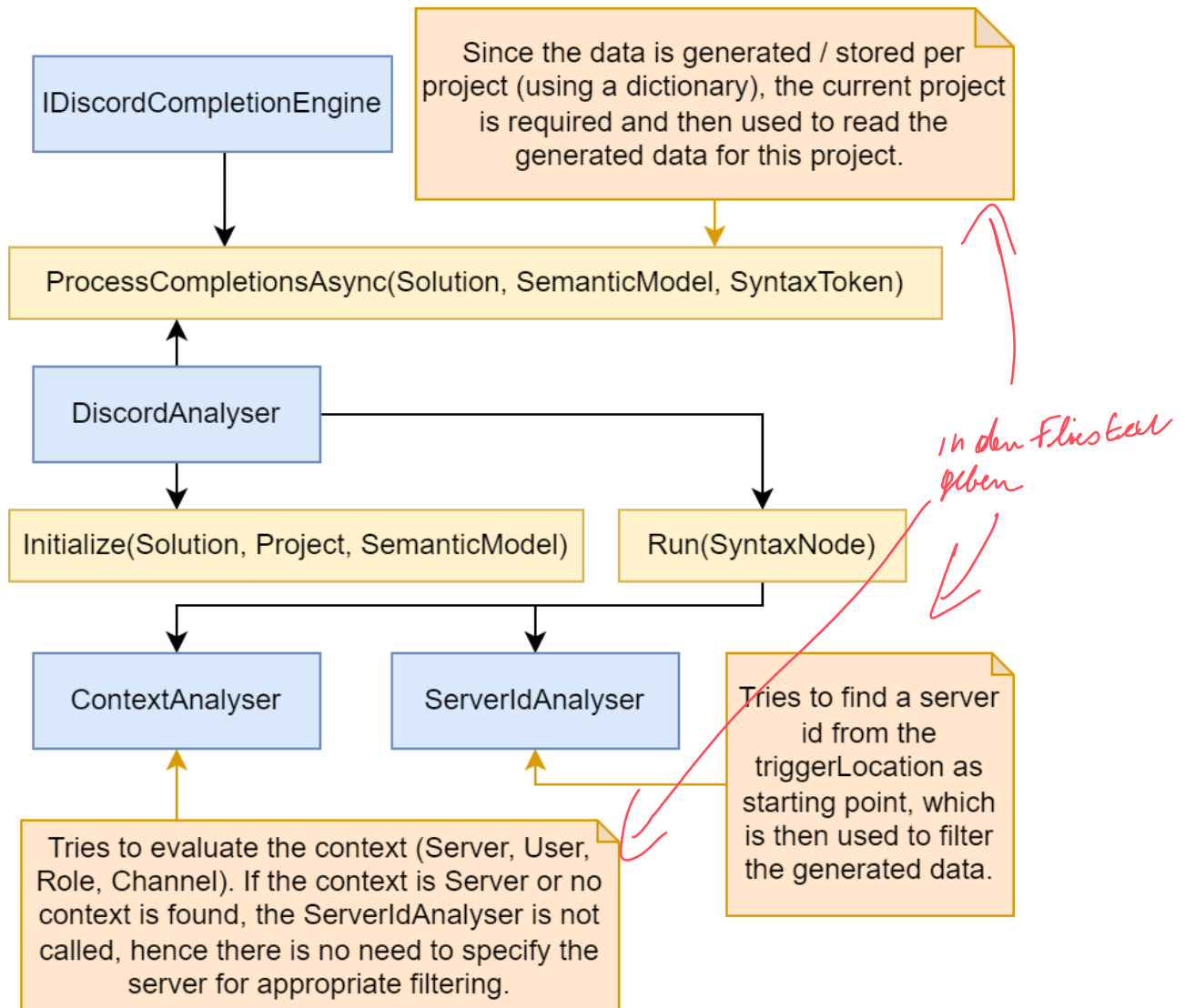


Figure 18: ~~Shows~~ analysis workflow with the IDiscordCompletionEngine as start point

Figure 18 resembles the analysis process which is triggered every time the editor requests IntelliSense. The *IDiscordCompletionEngine* creates the *DiscordAnalyser* that will first get the context, and then proceeds to get a server id if required. Since Roslyn maintains the compiler ~~api~~ (Figure 9), there is access to a big variety of classes that resemble different types of code.

Listing 28: Code snippets from the context analyser

```

1 ...
2 private async Task ResolveNode(SyntaxNode node) {
3     switch (node) {
4         case InvocationExpressionSyntax invocation:
5             CheckForContext(invocation);
6             break;

```

```

7      case BinaryExpressionSyntax binary when binary.Kind() ==
          SyntaxKind.EqualsExpression || binary.Kind() == SyntaxKind.
              NotEqualsExpression:
8          await ResolveNode(binary.Left);
9          break;
10     case MemberAccessExpressionSyntax memberAccess:
11         await ResolveNode(memberAccess.Expression);
12         break;
13     case ConditionalAccessExpressionSyntax conditionalAccess:
14         await ResolveNode(conditionalAccess.Expression);
15         break;
16     case IdentifierNameSyntax identifier:
17         CheckForContext(identifier);
18         break;
19     case SwitchStatementSyntax switchStatement:
20         await ResolveNode(switchStatement.Expression);
21         break;
22     case ArgumentSyntax argument:
23         await ResolveArgument(argument);
24         break;
25     case ArgumentListSyntax argumentList:
26         await ResolveNode(argumentList.Parent);
27         break;
28     }
29 }

```

For instance when taking a look at Listing 28, there are many different classes used that resemble a C# *SyntaxNode*, like an *InvocationExpressionSyntax* in line 4, which stands for a method call. When utilizing these features, the users code can be queried efficiently. But it is again important to state, that this whole analysis is called each time IntelliSense will trigger. To not negatively impact the users coding experience, it is important that the algorithms analysing code are fast. This can be more easily achieved by addressing termination conditions instead of rushing into development.

kommt in diesem Abschnitt sehr häufig vor

6.4 Potential result

Warum „Potential“?

Einführung

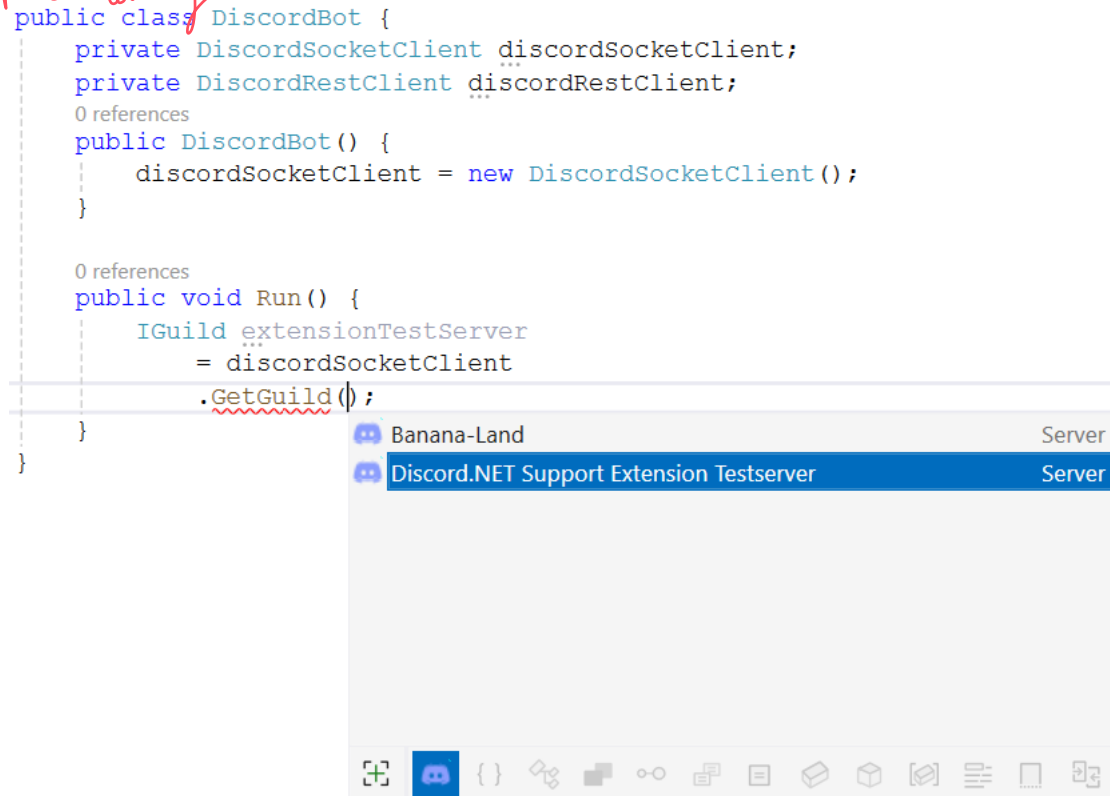


Figure 19: Completion items for getting a server

Figure 19 displays the possible completion items for this context. In this case, the found context is *Server*, so each server found in the generated data (Paragraph 6.1) is going to be provided as completion item. This action's responsible component *IAsyncCompletionSource* is mentioned in Listing 24.

```
public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public void Run() {
        IGuild extensionTestServer
            = discordSocketClient
                .GetGuild(1157751183184760953 /* Discord.NET Support Extension Testserver (Server) */);
    }
}
```

Figure 20: Committed server completion item

Figure 20 shows how the selected completion item is committed into the editor. This action's responsible component *IAsyncCompletionCommitManager* is mentioned

in Listing 26.

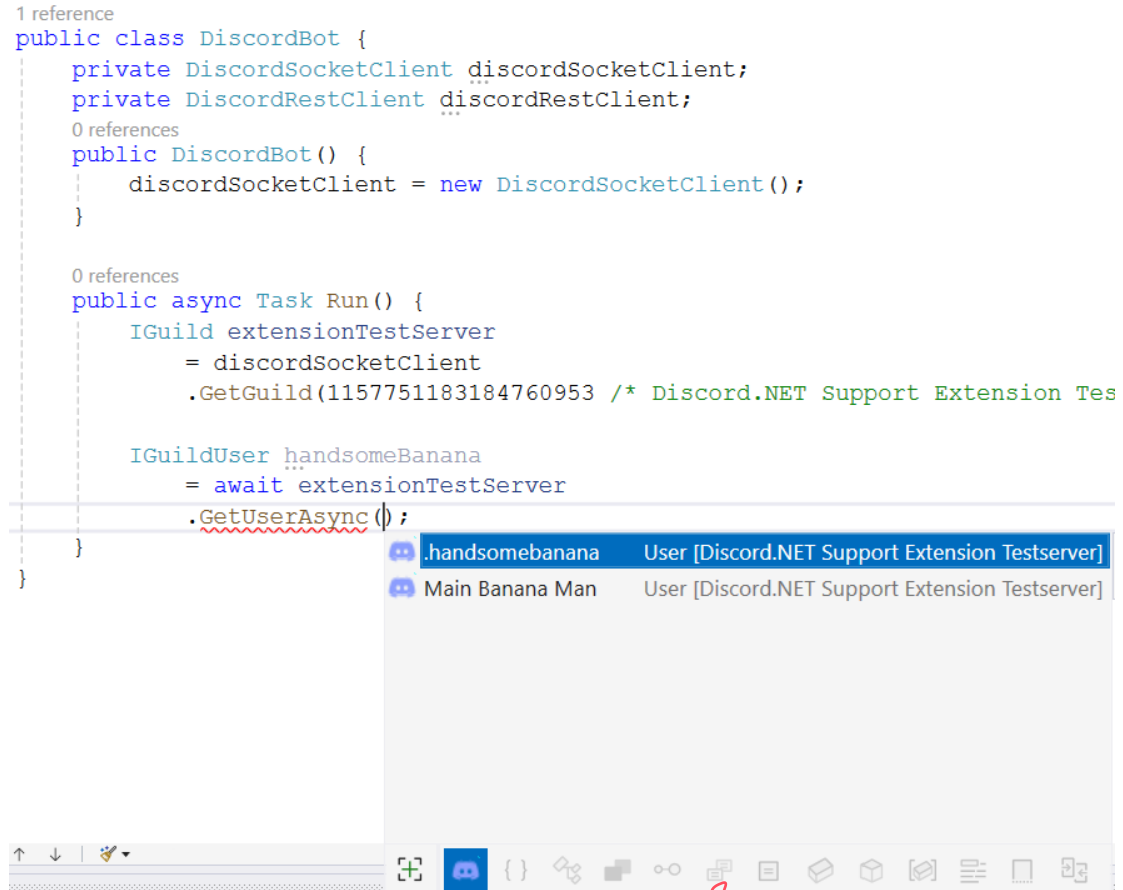


Figure 21: Completion items for getting a user that is inside the given server

It is important to note, that when the given context is not *Server*, like in this case *User*, not every user found in the generated data is taken and provided as completion item. The Server-Id analyser mentioned in Subsection 6.3 determines on which server-variable the *GetUserAsync* method is called. So for this example, when requesting the user completion items, the algorithm knows that this method is called on the *extension-TestServer* variable and will proceed to get the id connected with this variable, which is the static value inside the *GetGuild* method. If the algorithm is unable to find a server id, it will provide all users found in the generated data.

6 BUILDING THE VISUAL STUDIO EXTENSION

```

1 reference
public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public async Task Run() {
        IGuild extensionTestServer
        = discordSocketClient
        .GetGuild(1157751183184760953 /* Discord.NET Support Extension Testserver (Server) */);

        IGuildUser handsomeBanana
        = await extensionTestServer
        .GetUserAsync(270995861826306049 /* .handsomebanana (User [Discord.NET Support Extension Testserver]) */);
    }
}

```

Figure 22: Committed user completion item

Figure 22 shows how the selected completion item is committed into the editor. More complex scenarios like Linq *Linq expressions* are also supported.

```

public class DiscordBot {
    private DiscordSocketClient discordSocketClient;
    private DiscordRestClient discordRestClient;
    0 references
    public DiscordBot() {
        discordSocketClient = new DiscordSocketClient();
    }

    0 references
    public async Task Run() {
        SocketGuild extensionTestServer
        = discordSocketClient
        .GetGuild(1157751183184760953 /* Discord.NET Support Extension Testserver (Server) */);

        IRole role = extensionTestServer.Roles.FirstOrDefault(e => e.Id ==
    }
}

```

@everyone	Role [Discord.NET Support Extension Testserver]
Contributor	Role [Discord.NET Support Extension Testserver]
Main Banana Man	Role [Discord.NET Support Extension Testserver]
ReadOnly	Role [Discord.NET Support Extension Testserver]

Figure 23: Role suggestions in Linq method

As shown in Figure 23, when using Linq *Linq* methods the context is determined and the suggestions are provided correctly.

7 Closing

Creating the Visual Studio Extension for making Discord bots is a big leap towards solving common headaches for developers in this area. It hooks up with Discord.NET and uses Microsoft Roslyn to check and improve ^{an} your code, making this toolkit both powerful and easy to use for building bots. This thesis has shown how the extension can make coding faster, cut down on mistakes, and make the whole bot-building process more comfortable. Looking ahead, there's room to bring in more APIs, improve the user interface and the code checking features to help with even more development tasks.

↑
Das klingt eher nach einem Werbe Prospekt

Sie sollten die wichtigsten Ergebnisse der Arbeit zusammenfassen und diese auch bewerten (in welchem Ausmaß wurden die Ziele erreicht).

REFERENCES

References

Agnihotri, A., Gautam, H., and Singh, H. (2022). Discord bot.

Developer.com (2012). An introduction to microsoft roslyn.

Discord (2022). What is discord?

Discord.Net (2015). Discord.net.

GameInfoHub (2023). Inside the discord revolution: Insights into gaming's powerful communication platform.

Microsoft (2021a).

Microsoft (2021b). Understand the .net compiler platform sdk model.

Microsoft (2021c). Visual studio extensibility cookbook.

Microsoft (2021d). Work with a workspace.

Microsoft (2021e). Work with semantics.

Microsoft (2021f). Work with syntax.

Microsoft (2022). Explore code with the roslyn syntax visualizer in visual studio.

Microsoft (2023). Visual studio extension.

TheEnlightenedMindset (2023). Who invented discord? the story of its innovators and developers.

ToptalDevelopers (2022). How to make a discord bot: an overview and tutorial.

Vasani, M. (2017). *Roslyn Cookbook*. Packt Publishing Ltd.

Wieczorek, A. (2018a). Async completion api.

Wieczorek, A. (2018b). Async completion api.

Präzise bibliografische Angaben machen. Literaturverzeichnis besser aufbereiten.

groß u. kleinschreibung hier in der Quelle

→ für alle Referenzen überprüfen

Jede Quelle muss vom Leser auffindbar sein! Das ist absolute Grundvoraussetzung! Nach Möglichkeit auch den Autor angeben.